ASSIGNMENT 7: RESEARCH GAP ANALYSIS OF PUBLISHED WHITE PAPERS / ARTICLES

Mohd Talha Patrawala CMPN – B 23102B0025

1. Executive Summary & Research Gap Overview

This research gap analysis evaluates the thematic, methodological, and domain-specific evolution between two seminal 2026 cybersecurity reviews: *A Review of Cyber-Attacks on Web Applications and Their Countermeasures* (Beskri & Ng, Jan 2026) and *Bridging the Gap in Web API Security: A Systematic Review of Vulnerabilities, Misuse Patterns, and Developer Challenges* (Almjnoony et al., Jun 2026).

The fundamental **research gap** bridged across these studies reflects a critical shift in web defense paradigms—transitioning from **macro-level operational alignment** (evaluating how real-world breach data diverges from general web application academic literature) to **micro-level developer ergonomics and architectural specificity** (examining how API usability, documentation ambiguity, and cognitive friction lead software engineers to introduce recurring API security vulnerabilities).

2. Comparative Evolution Across Key Research Dimensions

Dimension	**Jan 2026 Paper (Beskri & Ng)**	**Jun 2026 Paper (Almjnoony et al.)**
Domain Scope	General Web Applications (Monolithic architecture, general web app layer).	Web APIs specifically (REST, SOAP, GraphQL protocols, & endpoint architecture).
Research Problem	Misalignment between academic literature attention and operational breach realities; patch management delays.	How API usability flaws, documentation ambiguity, and developer cognitive friction cause recurring API vulnerabilities.
Methodology	Case Enhanced-Systematic Literature Review (CS-SLR) combining academic papers and grey literature case studies.	PRISMA-2020 guided SLR with multi-dimensional taxonomy synthesis and 3-pillar conceptual perspective.
Analytical Metrics	Quantitative: Vulnerability Density (VD), Countermeasure Density (CRD), Real-World Density (RWD), Research Gap Formula.	Qualitative & Structural: Taxonomy mapping across Access Scope, Architectural Style, Purpose, and OWASP API Top 10.

Data Sources	32 Selected Academic Studies (2020–2024) + 34 Breach Case Studies / 54 Supporting Documents (2014–2024).	50 Peer-Reviewed Primary Academic Studies (2018–2023) + Industry Threat Reports (Akamai, Wallarm, Vaadata).
Primary Limitations	General app focus; lacks protocol-level API style breakdown; relies on third-party public breach reporting.	Excludes non-peer-reviewed grey literature breaches; lacks direct user-study testing with live developers in review.

3. Detailed Evolution of Research Problem and Focus

Jan 2026 Paper (Beskri & Ng): Operational Misalignment in Web Security

The January 2026 paper identifies a major disconnect between theoretical cybersecurity literature and operational security breaches. Standard literature reviews predominantly focused on front-end injection attacks—such as Cross-Site Scripting (XSS) and SQL Injection (SQLi)—in isolated lab settings. Meanwhile, real-world breaches were overwhelmingly driven by broken access control, security misconfigurations, and software supply chain failures. Furthermore, Beskri & Ng address the organizational delay in applying security patches, evaluating why enterprise remediation timelines lag significantly behind active threat disclosure.

Jun 2026 Paper (Almjnoony et al.): Human Factors & API Developer Ergonomics

The June 2026 study narrows its lens directly onto Web APIs (HTTP/HTTPS protocols) and shifts the research question from macro-level organizational patch delays to micro-level developer behavior. Almjnoony et al. investigate why API vulnerabilities recur despite well-established defensive frameworks (such as OWASP API Top 10). They identify that API security failures stem from poor API usability, unclear documentation, overly complex authentication workflows (e.g., OAuth 2.0 / OpenID Connect), and cognitive friction that causes software engineers to adopt insecure workarounds.

4. Methodological Progression & Analytical Frameworks

Case Enhanced-Systematic Literature Review (CS-SLR) vs. PRISMA-2020 SLR

Beskri & Ng pioneered the CS-SLR methodology, inspired by Multivocal Literature Reviews (MLR). This approach synthesizes peer-reviewed academic literature (White Literature) alongside real-world incident case studies and industry breach reports (Grey Literature) through a rigorous dual Quality Assessment (QA) process.

To measure research gaps empirically, Beskri & Ng construct quantitative density metrics:

$$\text{Vulnerability Density (VD)} = N_{\text{mentions}} / T_{\text{studies}}$$
$$\text{Real-World Density (RWD)} = N_{\text{incidents}} / T_{\text{incidents}}$$
$$\text{Research Gap} = \text{RWD} - \text{VD}$$

In contrast, Almjnoony et al. utilize a standard PRISMA-2020 guided SLR protocol combined with a comprehensive taxonomy-driven synthesis. Rather than quantitative gap scoring, Almjnoony et al. categorize Web API security along three structured functional pillars:

- **Access Scope Taxonomy:** Categorizing endpoints into Open/Public, Private, Partner, and Composite APIs.
- **Architectural Style Breakdown:** Analyzing protocol-specific attack vectors across REST, SOAP, and GraphQL interfaces.
- **3-Pillar Integrative Model:** Connecting Developer Behavior, Web API Usability, and Security Tool Effectiveness to uncover structural failure modes.

5. Data Corpus, Modalities & Protocol Specifics

General Web Vulnerabilities vs. Protocol-Level API Vectors

The January 2026 paper evaluates web applications in aggregate using the OWASP Top 10 (2025) framework, focusing on XSS, SQLi, CSRF, Path Traversal, and File Upload vulnerabilities. Its dataset comprises 32 academic studies and 34 high-profile incident case studies (including Capital One, Equifax, Marriott, and British Airways).

The June 2026 paper expands data modalities into modern API architectural styles, drawing from 50 primary peer-reviewed studies (2018–2023) and mapped against the OWASP API Security Top 10 (2023):

- **REST APIs (HTTP/JSON):** Broken Object Level Authorization (BOLA), Broken Object Property Level Authorization (BOPLA), endpoint overexposure, and token lifecycle misconfigurations.
- **SOAP APIs (XML/HTTP):** XML External Entity (XXE) injection, improper XML parser settings, and complex WS-Security specification implementation errors.
- **GraphQL APIs:** Single-endpoint query depth and complexity of DoS attacks, field-level authorization bypasses, and accidental introspection exposure in production environments.

6. Comparative Limitations and Future Research Directions

Beskri & Ng (Jan 2026) acknowledge that relying on third-party breach reporting and media disclosures in grey literature can obscure exact technical root causes. Furthermore, their high-level operational focus leaves unaddressed the micro-architectural differences among modern API protocols.

Almjnoony et al. (Jun 2026) exclude non-peer-reviewed grey literature breaches from their systematic selection pool to maintain academic rigor, which risks missing newly emerging zero-day API exploit trends. Additionally, while they identify developer cognitive load as a primary driver of API vulnerabilities, their review relies on secondary observational literature rather than direct empirical user-testing with live software developers.

Synthesis: Together, these two studies demonstrate a clear maturation in web security literature—moving from empirical validation that academic research was misaligned with operational breaches (Jan

2026) to establishing developer usability, interface design, and tool friction as the primary root causes of recurring API security failures (Jun 2026).